Diabetes Progression Regression

Applied AI/ML Mini Project report | CHRIST (Deemed to be University), Department of AI and Data Science Engineering

Student name	Rachel Elizabeth Joy
Register number	2667213
Email	rachel.elizabeth@mtech.christuniversity.in
Course	**Google DeepMind Research**
Problem ID and title	17 – Diabetes Progression Regression
Dataset and source	scikit-learn: sklearn.datasets.load_diabetes (Efron et al., 2004)
Colab notebook link	https://colab.research.google.com/drive/1b750JkYeiisxDnfrFAzUkW90f1I-YSvb
Date of submission	27 Oct 2026

Declaration

I declare that this report and the notebook linked above are my own work. Sources and tools I used are listed in the report. I can explain all the code and results in it.

Signature / typed name: Rachel Elizabeth Joy Date: 27 Oct 2026

Abstract

This project predicts a quantitative measure of diabetes disease progression one year after baseline for 442 patients, using ten baseline measurements from the scikit-learn diabetes dataset. After exploratory analysis, the data were split 60/20/20 into training, validation and test sets, and all imputation, encoding, scaling and two engineered features were fitted inside a pipeline on training data only. A mean-predictor baseline was compared with ordinary least squares, Ridge, Lasso and ElasticNet (tuned with repeated 5-fold cross-validation), a random forest and gradient boosting. The regularised linear models performed best; Lasso ($\alpha = 0.398$) was selected and reached a test RMSE of 52.2, MAE of 41.6 and R^2 of 0.548, against a baseline RMSE of 77.7. Coefficient paths show that Ridge shrinks all twelve coefficients smoothly, while Lasso drops features one at a time and keeps only the obesity–blood pressure interaction and serum triglycerides at strong regularisation.

1. Problem statement and objective

The task is a supervised regression problem. For each patient we predict a single number, a quantitative measure of diabetes disease progression recorded one year after a baseline visit, from ten measurements taken at that visit: age, sex, body mass index (BMI), average blood pressure and six blood serum values. The target is not a diagnosis; it is a continuous score that runs from 25 to 346, where higher values mean faster progression.

A model like this could help a diabetes clinic or a care-planning team rank newly seen patients by expected risk, so that those likely to deteriorate fastest are offered earlier follow-up, dietary support or a medication review. It could also tell researchers which baseline measurements carry the most information about progression.

The main metric is the **root mean squared error (RMSE)**. It is measured in the same units as the progression score, so it can be read directly as a typical size of error, and because it squares errors it penalises large misses more heavily, which matters when a badly under-predicted patient could miss timely care. MAE is reported alongside it as a more robust average error, and R^2 shows the share of variance explained compared with simply predicting the mean. Success means a clear improvement over the mean baseline on data the model has never seen.

2. Dataset description

The data come from the study by Efron et al. (2004) and are shipped with scikit-learn as *load_diabetes* [1], [6]. Loading with *scaled=False* gives the original clinical units, so that scaling can be learnt inside the pipeline instead of being applied to the full dataset in advance. The serum columns s1 to s6 were renamed to their clinical meaning (Table 1).

There are **442 rows and 11 columns**: ten features and the target. All columns are stored as float64 and there are **no missing values** (Figure 1). Sex is a binary category coded 1 and 2, with 235 and 207 patients respectively. The target has a mean of 152.1, a median of 140.5 and a standard deviation of 77.1, so a model that always predicts the mean would be off by about 77 points on average. The dataset is small, which strongly shapes the choices made later: complex models have little data to learn from, and every split decision changes results noticeably, so cross-validation is repeated.

Table 1: Features used in the model

Feature	Type	Meaning	Missing values
age	numeric	Age in years	0
sex	categorical	Sex, coded 1 / 2	0
bmi	numeric	Body mass index	0
bp	numeric	Average blood pressure	0
tc (s1)	numeric	Total serum cholesterol	0
ldl (s2)	numeric	Low-density lipoproteins	0
hdl (s3)	numeric	High-density lipoproteins	0
tch (s4)	numeric	Total cholesterol / HDL	0
ltg (s5)	numeric	Log of serum triglycerides	0

Feature	Type	Meaning	Missing values
glu (s6)	numeric	Blood sugar level	0
target	numeric	Progression score after one year	0

```
[2]  raw = load_diabetes(as_frame=True, scaled=False)
     data = raw.frame.rename(columns={'s1': 'tc', 's2': 'ldl', 's3': 'hdl',
                                      's4': 'tch', 's5': 'ltg', 's6': 'glu'})
     print('Shape:', data.shape)
     display(data.head())
     data.info()
```

```
Shape: (442, 11)

    age  sex   bmi     bp     tc    ldl   hdl  tch     ltg   glu  target
0  59.0  2.0  32.1  101.0  157.0   93.2  38.0  4.0  4.8598  87.0   151.0
1  48.0  1.0  21.6   87.0  183.0  103.2  70.0  3.0  3.8918  69.0    75.0
2  72.0  2.0  30.5   93.0  156.0   93.6  41.0  4.0  4.6728  85.0   141.0
3  24.0  1.0  25.3   84.0  198.0  131.4  40.0  5.0  4.8903  89.0   206.0
4  50.0  1.0  23.0  101.0  192.0  125.4  52.0  4.0  4.2905  80.0   135.0

<class 'pandas.DataFrame'>
RangeIndex: 442 entries, 0 to 441
Data columns (total 11 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   age     442 non-null    float64
 1   sex     442 non-null    float64
 2   bmi     442 non-null    float64
 3   bp      442 non-null    float64
 4   tc      442 non-null    float64
 5   ldl     442 non-null    float64
 6   hdl     442 non-null    float64
 7   tch     442 non-null    float64
 8   ltg     442 non-null    float64
 9   glu     442 non-null    float64
 10  target  442 non-null    float64
dtypes: float64(11)
memory usage: 38.1 KB
```

Figure 1: Loading the data in original units, data.head() and data.info(): 442 rows, 11 float columns, no nulls

Figure 1 confirms the shape, the renamed columns and that every column is complete, so no row has to be dropped.

3. Exploratory data analysis

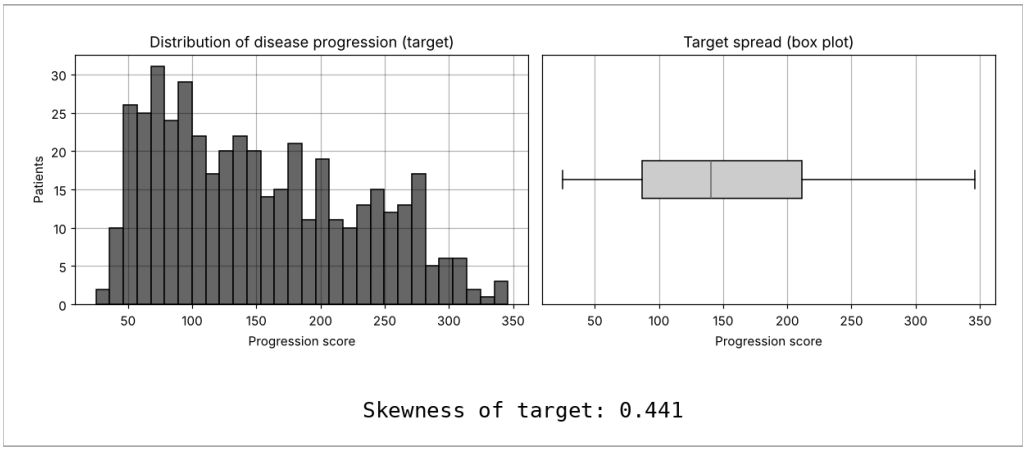


Figure 2: Histogram and box plot of the target, with its skewness

Figure 2 shows that progression is right-skewed (skewness 0.44): most patients score between 87 and 212, with a long tail towards 346 and no extreme outliers beyond the whiskers. The skew is mild enough that a transformation of the target was not needed.

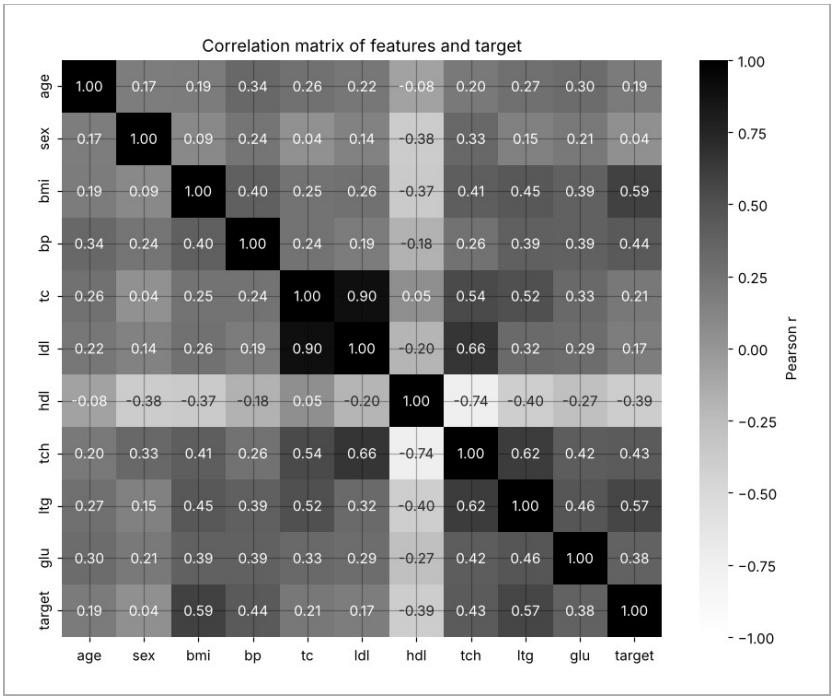


Figure 3: Correlation matrix of all features and the target (greyscale heatmap)

In Figure 3, the features most correlated with progression are **bmi (r = 0.59)**, **ltg (0.57)** and **bp (0.44)**, while **hdl** is negatively correlated (−0.40), which fits its role as "good" cholesterol. Several features are strongly correlated with each other: tc and ldl (r = 0.90), and hdl and tch (−0.74).

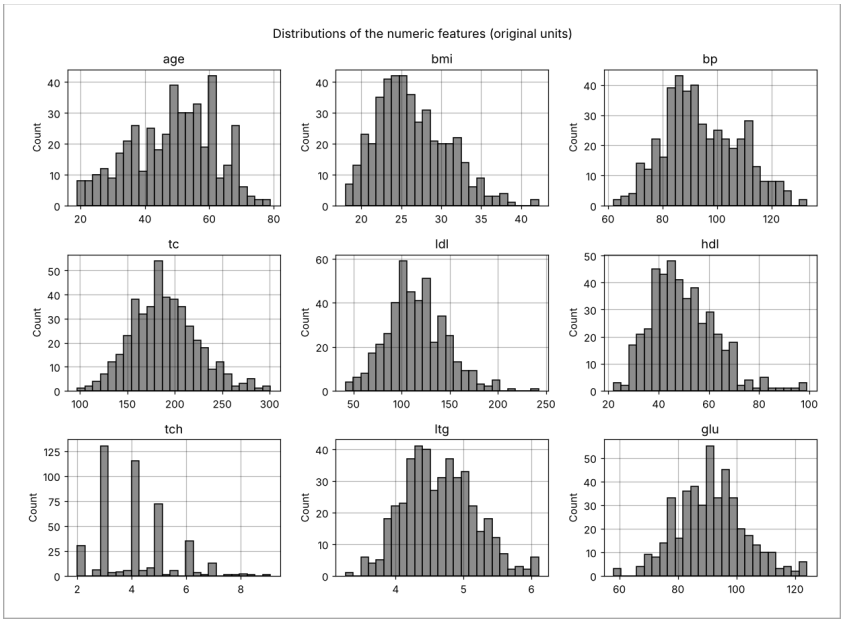


Figure 4: Distributions of the nine numeric features in original units

Figure 4 shows that the features sit on very different scales, from ltg around 3 to 6 to tc around 100 to 300. This is why scaling is essential before regularisation: Ridge and Lasso penalise

coefficient size, so without scaling the penalty would fall unevenly. Most features are roughly bell-shaped; tch is discrete because it was rounded in the original study.

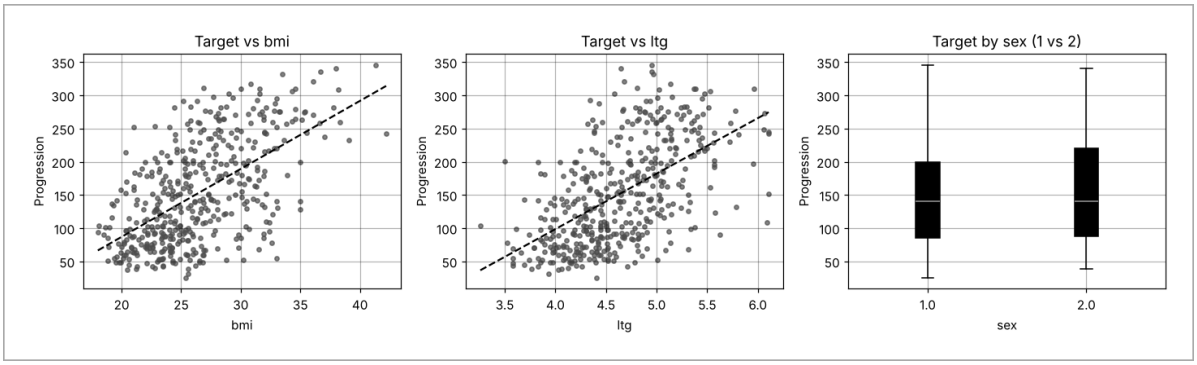


Figure 5: Target against bmi and ltg with a fitted line, and target by sex

Figure 5 explains the target best: progression rises steadily with both bmi and ltg, but the scatter around each line is wide, so no single measurement predicts it well and an R^2 well below 1 should be expected. The two sexes have similar medians, so sex alone carries little signal.

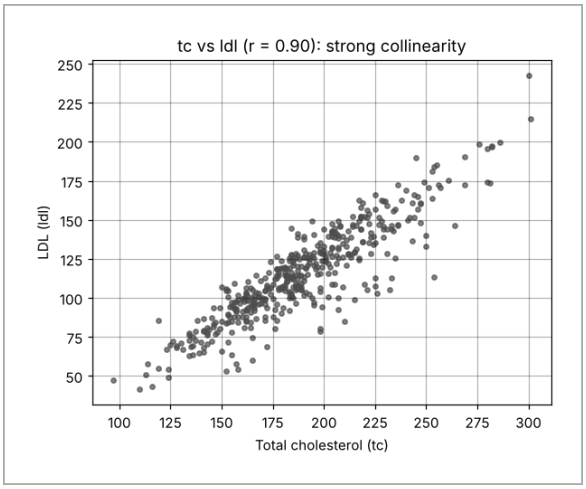


Figure 6: Total cholesterol against LDL, showing near-collinearity (r = 0.90)

Figure 6 shows the collinearity directly. When two inputs move together, ordinary least squares can give them large coefficients of opposite sign that cancel out. This is the behaviour the focus requirement examines, because Ridge, Lasso and ElasticNet each resolve it in a different way.

4. Data preparation and feature engineering

The data were split **before** any fitting. A first split held out 20% as the test set, and the remainder was split again 75/25 into training and validation sets, giving **264 / 89 / 89 rows** (Figure 8). Because the target is continuous, the splits were stratified on target quintiles, so each set covers the full range of progression; the target means of the three sets are 152.3, 150.8 and 152.9. The seed is fixed at 42 throughout.

All other steps sit inside one scikit-learn Pipeline (Figure 7), so they are learnt on the training folds only and re-applied, unchanged, to validation and test data. This prevents leakage of test information into the scaler. Two features were engineered from clinical reasoning. **bmi_x_bp** multiplies BMI by blood pressure to represent combined metabolic load, since obesity and

hypertension tend to worsen each other; it is correlated 0.63 with the target on the training set, more strongly than either input alone. **ldl_hdl_ratio** is a standard lipid-risk ratio (correlation 0.26). No outliers were removed: the box plots showed none that looked like recording errors, and with 442 patients each row is valuable.

Table 2: Data preparation steps

Step	What I did	Why
Missing values	Median imputer (numeric), most-frequent imputer (sex), inside the pipeline	No values are missing now, but the pipeline stays safe for new patients with gaps
Encoding	One-hot encoding of sex with the first level dropped	Sex is a category, not a quantity; dropping one level avoids a redundant column
Scaling	StandardScaler on all numeric and engineered features	Regularisation penalises coefficient size, so all features must be on one scale
New features	bmi_x_bp, ldl_hdl_ratio	Combined metabolic load and lipid risk ratio, both clinically motivated
Split	264 train / 89 validation / 89 test, stratified on target quintiles, seed 42	Test set kept untouched; stratification keeps the target range similar

```
[10]  def add_features(df):
          df = df.copy()
          df['bmi_x_bp'] = df['bmi'] * df['bp']          # combined metabolic load: obesity x blood pressure
          df['ldl_hdl_ratio'] = df['ldl'] / df['hdl']    # clinical lipid risk ratio
          return df

      num_cols = ['age', 'bmi', 'bp', 'tc', 'ldl', 'hdl', 'tch', 'ltg', 'glu', 'bmi_x_bp', 'ldl_hdl_ratio']
      cat_cols = ['sex']
      pre = ColumnTransformer([
          ('num', Pipeline([('impute', SimpleImputer(strategy='median')), ('scale', StandardScaler())]), num_cols),
          ('cat', Pipeline([('impute', SimpleImputer(strategy='most_frequent')),
                            ('onehot', OneHotEncoder(drop='first', handle_unknown='ignore'))]), cat_cols)])
      feat_eng = FunctionTransformer(add_features)
      def make_pipe(model):
          return Pipeline([('features', feat_eng), ('prep', pre), ('model', model)])
      print(make_pipe(Ridge()))
```

Figure 7: Feature engineering function and the preprocessing ColumnTransformer inside the pipeline

```
[9]   X = data.drop(columns='target'); y = data['target']
      # 60 / 20 / 20 split, stratified on target quintiles so all splits cover the full range
      bins = pd.qcut(y, 5, labels=False)
      X_tmp, X_test, y_tmp, y_test = train_test_split(X, y, test_size=0.20, random_state=SEED, stratify=bins)
      bins_tmp = pd.qcut(y_tmp, 5, labels=False)
      X_train, X_val, y_train, y_val = train_test_split(X_tmp, y_tmp, test_size=0.25, random_state=SEED, stratify=b
          ins_tmp)
      for n, (a, b) in {'Train': (X_train, y_train), 'Validation': (X_val, y_val), 'Test': (X_test, y_test)}.items(
          ):
          print(f'{n:<11} X={a.shape}  y={b.shape}  target mean={b.mean():6.1f}')
```

```
Train       X=(264, 10)  y=(264,)  target mean= 152.3
Validation  X=(89, 10)  y=(89,)  target mean= 150.8
Test        X=(89, 10)  y=(89,)  target mean= 152.9
```

Figure 8: Stratified 60/20/20 split and the resulting set sizes and target means

5. Models and experimental setup

Seven models were compared, all wrapped in the same pipeline so that preprocessing was identical (Figure 9). The **baseline** is a DummyRegressor that always predicts the training mean.

Ordinary least squares (OLS) is the unregularised linear reference. **Ridge** adds an L2 penalty that shrinks all coefficients towards zero [3]; **Lasso** adds an L1 penalty that can set coefficients exactly to zero [4]; **ElasticNet** mixes the two through l1_ratio [5]. Two tree ensembles, a **random forest** [7] and **gradient boosting** [8], test whether non-linear models add anything on so few rows.

Tuning used **5-fold cross-validation repeated 3 times** (15 fits per setting) on the training set only, with negative RMSE as the score. Repetition was used because single 5-fold estimates vary by several points on 264 rows. Ridge was searched over 61 values of α from 10^{-3} to 10^{3}, Lasso over 51 values from 10^{-3} to 10^{2}, and ElasticNet over 26 values of α crossed with five l1_ratio values (Figure 9). The ensembles were tuned with a 25-draw randomised search over depth, number of trees, leaf size, feature fraction and, for boosting, learning rate and subsampling (Figure 10). The same RepeatedKFold object with seed 42 was used for every model, so all were compared on identical folds.

The validation set played a separate role. It was never used for tuning, only as a second, independent check after cross-validation, so that a model that happened to fit the training folds well could be caught before the test set was opened. Model selection followed a rule fixed in advance: lowest cross-validated RMSE among the learnt models, with validation RMSE used only to confirm the ranking. This matters because the gaps between the best models are small, and choosing on the test set would make the reported score optimistic.

Table 3: Models, chosen settings and cross-validated RMSE (training set)

Model	**Key settings (after tuning)**	**CV RMSE**	**Notes**
Baseline: mean	Predicts training mean	76.72 ± 5.57	Floor to beat
OLS	No penalty	55.69 ± 4.88	Unstable coefficients
Ridge	α = 19.95	55.45 ± 4.83	Keeps all features
Lasso	α = 0.398	55.32 ± 5.04	Lowest CV RMSE; 9 features
ElasticNet	α = 0.398, l1_ratio = 0.9	55.39 ± 4.95	Close to Lasso
Random forest	200 trees, leaf 5, max_features 0.33	56.71 ± 4.91	No gain over linear
Gradient boosting	400 trees, lr 0.01, depth 2, subsample 0.7	57.01 ± 5.93	Most variable

```
[12]  cv = RepeatedKFold(n_splits=5, n_repeats=3, random_state=SEED)
      alphas = np.logspace(-3, 3, 61)
      grids = {
          'Ridge':      (Ridge(random_state=SEED),                    {'model__alpha': alphas}),
          'Lasso':      (Lasso(max_iter=50000, random_state=SEED),    {'model__alpha': np.logspace(-3, 2, 51)}),
          'ElasticNet': (ElasticNet(max_iter=50000, random_state=SEED),
                         {'model__alpha': np.logspace(-3, 2, 26), 'model__l1_ratio': [0.1, 0.3, 0.5, 0.7, 0.9]}),
      }
      tuned = {}
      for name, (est, grid) in grids.items():
          gs = GridSearchCV(make_pipe(est), grid, cv=cv, scoring='neg_root_mean_squared_error', n_jobs=-1)
          gs.fit(X_train, y_train); tuned[name] = gs
          print(f'{name:<11} best {gs.best_params_}  CV RMSE = {-gs.best_score_:.2f}')
```

```
Ridge       best {'model__alpha': np.float64(19.952623149688787)}  CV RMSE = 55.44

Lasso       best {'model__alpha': np.float64(0.3981071705534973)}  CV RMSE = 55.32

ElasticNet  best {'model__alpha': np.float64(0.3981071705534973), 'model__l1_ratio': 0.9}  CV RMSE = 55.39
```

Figure 9: Repeated K-fold set-up and grid search for Ridge, Lasso and ElasticNet with the best settings

```
[13]  rf_grid = {'model__n_estimators': [200, 400], 'model__max_depth': [3, 5, 8, None],
                 'model__min_samples_leaf': [2, 5, 10], 'model__max_features': [0.33, 0.5, 1.0]}
      gb_grid = {'model__n_estimators': [100, 200, 400], 'model__learning_rate': [0.01, 0.03, 0.1],
                 'model__max_depth': [2, 3], 'model__subsample': [0.7, 1.0], 'model__min_samples_leaf': [5, 10]}
      for name, est, grid in [('RandomForest', RandomForestRegressor(random_state=SEED), rf_grid),
                              ('GradBoost', GradientBoostingRegressor(random_state=SEED), gb_grid)]:
          rs = RandomizedSearchCV(make_pipe(est), grid, n_iter=25, cv=cv, random_state=SEED,
                                  scoring='neg_root_mean_squared_error', n_jobs=-1)
          rs.fit(X_train, y_train); tuned[name] = rs
          print(f'{name:<12} best {rs.best_params_}  CV RMSE = {-rs.best_score_:.2f}')
```

```
RandomForest best {'model__n_estimators': 200, 'model__min_samples_leaf': 5, 'model__max_features': 0.33, 'mode
    l__max_depth': None}  CV RMSE = 56.70

GradBoost    best {'model__subsample': 0.7, 'model__n_estimators': 400, 'model__min_samples_leaf': 10, 'model__
    max_depth': 2, 'model__learning_rate': 0.01}  CV RMSE = 57.01
```

Figure 10: Randomised search for the random forest and gradient boosting

6. Results

Table 4 and Figure 11 compare every model on the same cross-validation folds and on the same validation set; the test set was not used for any of these choices. All learnt models clearly beat the baseline, cutting RMSE from about 77 to about 56. The three regularised linear models are best and lie within 0.1 of each other in cross-validated RMSE, well inside one standard deviation (about 5), so they are effectively tied. The tree ensembles are slightly worse, which suggests that the relationship is mostly linear once the bmi × bp interaction is supplied, and that 264 rows are too few for trees to find anything extra (Figure 12).

Table 4: Model comparison on cross-validation and the validation set

Model	**CV RMSE**	**Val RMSE**	**Val MAE**	**Val R^2**
Baseline (mean)	76.72	77.75	65.64	0.000
OLS	55.69	56.49	46.28	0.472
Ridge	55.45	55.84	45.31	0.484
Lasso	55.32	55.88	45.44	0.483
ElasticNet	55.39	55.79	45.40	0.485
Random forest	56.71	57.06	46.57	0.461

Model	CV RMSE	Val RMSE	Val MAE	Val R^2
Gradient boosting	57.01	56.91	46.41	0.464
Lasso – final, TEST set	—	52.21	41.63	0.548
Baseline – TEST set	—	77.65	—	—

```
[14]  models = {'Baseline (mean)': make_pipe(DummyRegressor()), 'OLS': make_pipe(LinearRegression())}
      models.update({k: v.best_estimator_ for k, v in tuned.items()})
      rows = []
      for name, m in models.items():
          cvres = cross_validate(m, X_train, y_train, cv=cv, scoring='neg_root_mean_squared_error')
          m.fit(X_train, y_train); p = m.predict(X_val)
          rows.append({'Model': name, 'CV RMSE': -cvres['test_score'].mean(), 'CV RMSE sd': cvres['test_score'].std
          (),
                       'Val RMSE': mean_squared_error(y_val, p) ** 0.5, 'Val MAE': mean_absolute_error(y_val, p),
                       'Val R2': r2_score(y_val, p)})
      results = pd.DataFrame(rows).set_index('Model').round(3)
      display(results)
```

Model	CV RMSE	CV RMSE sd	Val RMSE	Val MAE	Val R2
Baseline (mean)	76.720	5.565	77.749	65.636	-0.000
OLS	55.694	4.875	56.488	46.277	0.472
Ridge	55.445	4.830	55.842	45.311	0.484
Lasso	55.323	5.042	55.884	45.440	0.483
ElasticNet	55.391	4.952	55.792	45.395	0.485
RandomForest	56.705	4.913	57.058	46.566	0.461
GradBoost	57.013	5.925	56.905	46.409	0.464

Figure 11: Evaluation loop over all models and the results table printed in the notebook

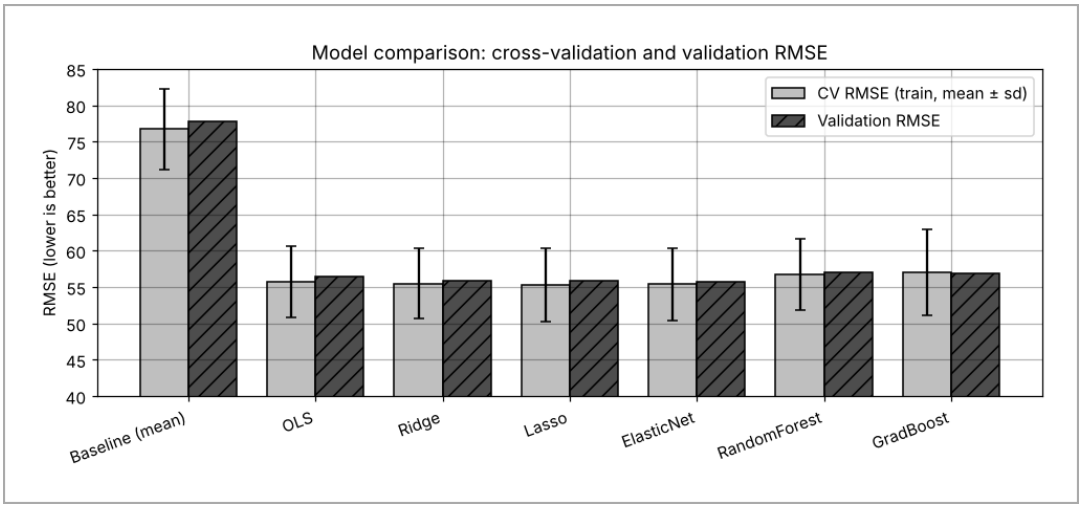


Figure 12: Cross-validated RMSE (mean ± sd) and validation RMSE for every model

Lasso was chosen because it has the lowest cross-validated RMSE, the most stable estimate available, and because it gives a simpler model with three coefficients set to zero. It was refit on training plus validation data and scored **once** on the test set (Figure 13): **RMSE 52.21, MAE 41.63 and R^2 0.548**, against a baseline RMSE of 77.65. The test RMSE is slightly lower than the CV estimate, which is within normal fold-to-fold variation for 89 patients rather than a sign of leakage.

```
[16]  lin = results.drop(index=['Baseline (mean)'])
      best_name = lin['CV RMSE'].idxmin()
      print('Selected model:', best_name)
      final = models[best_name]
      X_trval, y_trval = pd.concat([X_train, X_val]), pd.concat([y_train, y_val])
      final.fit(X_trval, y_trval)
      p_test = final.predict(X_test)
      test_scores = {'Test RMSE': mean_squared_error(y_test, p_test) ** 0.5,
                     'Test MAE': mean_absolute_error(y_test, p_test), 'Test R2': r2_score(y_test, p_test)}
      base_test = make_pipe(DummyRegressor()).fit(X_trval, y_trval).predict(X_test)
      print({k: round(v, 3) for k, v in test_scores.items()})
      print('Baseline test RMSE for reference:', round(mean_squared_error(y_test, base_test) ** 0.5, 3))
```

```
Selected model: Lasso
{'Test RMSE': 52.213, 'Test MAE': 41.63, 'Test R2': 0.548}
Baseline test RMSE for reference: 77.651
```

Figure 13: Selecting the model, refitting on train + validation and the single test-set evaluation

7. Focus requirement

Requirement: *Compare Ridge, Lasso and ElasticNet and show how coefficients shrink as regularisation grows.*

To isolate the effect of the penalty, the preprocessing pipeline was fitted on the training set once, and each model was refit on the same standardised matrix across 80 values of α from 10^{-3} to 10^{3}. ElasticNet used the tuned l1_ratio of 0.9. The coefficients at each model's CV-chosen α were tabulated against OLS (Figure 14), the full paths were plotted (Figure 15), and the overall size and sparsity of the coefficient vector were tracked (Figure 16).

```
[17]  prep_only = Pipeline([('features', feat_eng), ('prep', pre)]).fit(X_train)
      Xs = prep_only.transform(X_train)
      l1r = tuned['ElasticNet'].best_params_['model__l1_ratio']
      path_alphas = np.logspace(-3, 3, 80)
      makers = {'Ridge': lambda a: Ridge(alpha=a), 'Lasso': lambda a: Lasso(alpha=a, max_iter=50000),
                f'ElasticNet (l1_ratio={l1r})': lambda a: ElasticNet(alpha=a, l1_ratio=l1r, max_iter=50000)}
      paths = {k: np.array([f(a).fit(Xs, y_train).coef_ for a in path_alphas]) for k, f in makers.items()}
      best_alpha = {k: tuned[k.split()[0]].best_params_['model__alpha'] for k in makers}
      coef_at_best = pd.DataFrame({k: f(best_alpha[k]).fit(Xs, y_train).coef_ for k, f in makers.items()},
                                  index=feature_names)
      coef_at_best['OLS'] = LinearRegression().fit(Xs, y_train).coef_
      display(coef_at_best.round(2))
      print('Non-zero coefficients:', (coef_at_best.abs() > 1e-6).sum().to_dict())
```

	Ridge	Lasso	ElasticNet (l1_ratio=0.9)	OLS
age	2.30	2.19	2.03	3.90
bmi	12.92	0.00	9.93	-39.51
bp	4.17	-6.76	1.10	-41.60
tc	-0.96	-3.59	-1.73	-22.80
ldl	-2.76	-0.00	-1.99	21.17
hdl	-9.22	-8.92	-8.55	-6.25
tch	0.26	-0.00	-0.00	-2.00
ltg	23.44	26.04	24.49	34.08
glu	3.65	3.06	3.23	4.07
bmi_x_bp	19.54	40.34	24.68	101.96
ldl_hdl_ratio	-5.00	-5.35	-4.44	-9.53
sex_2	-12.33	-15.16	-12.48	-19.05

```
Non-zero coefficients: {'Ridge': 12, 'Lasso': 9, 'ElasticNet (l1_ratio=0.9)': 11, 'OLS': 12}
```

Figure 14: Fitting coefficient paths and the coefficients at each CV-chosen α compared with OLS

The OLS column in Figure 14 shows the problem regularisation solves. Because bmi, bp and bmi_x_bp are strongly related, OLS gives bmi_x_bp a coefficient of 102 while giving bmi and bp large *negative* coefficients (about −40 each), which make no clinical sense. At their tuned α, all three penalised models remove these sign flips.

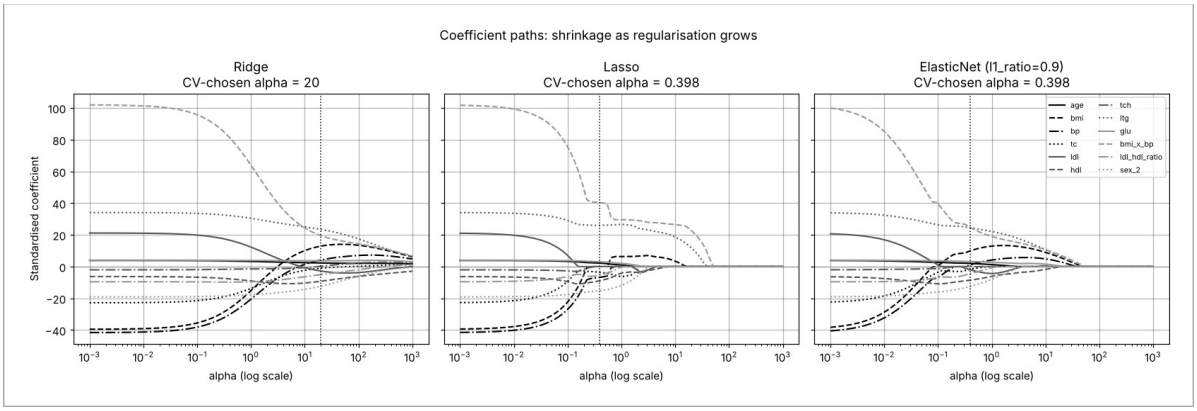


Figure 15: Coefficient paths of Ridge, Lasso and ElasticNet against α (log scale); dotted line marks the CV-chosen α

Figure 15 shows the three different shrinkage patterns. **Ridge** shrinks every coefficient smoothly and proportionally; none reaches zero, so all twelve are kept even at α = 1000, and correlated features end up sharing weight (bmi 12.9, bmi_x_bp 19.5). **Lasso** shrinks in steps and drives features to exactly zero one at a time. At its chosen α of 0.398 it has already removed bmi, ldl and tch, because bmi_x_bp and tc already carry that information. **ElasticNet** behaves in between: with l1_ratio 0.9 its paths look like Lasso but are smoother, and at the chosen α it drops only tch and keeps bmi (9.9).

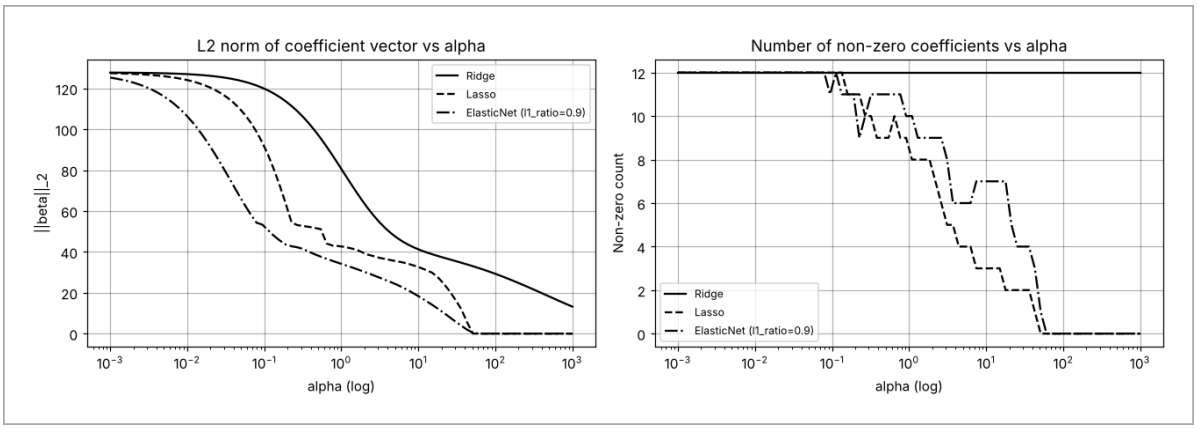


Figure 16: L2 norm of the coefficient vector and number of non-zero coefficients as α grows

```
Lasso: alpha beyond which each coefficient stays exactly zero (earliest at top)
Zero at the CV-chosen alpha: ['bmi', 'ldl', 'tch']
tch              0.382
bp               0.769
ldl_hdl_ratio     1.09
age               1.84
tc                 2.2
glu               2.62
sex_2             3.12
ldl               4.42
hdl               7.47
bmi               17.9
ltg               42.9
bmi_x_bp          51.2

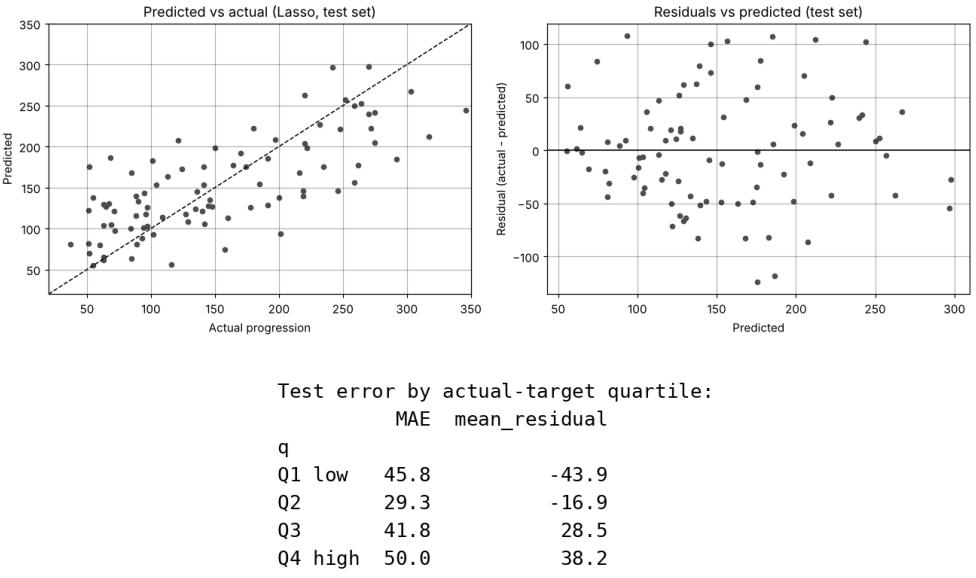
Lasso CV RMSE at selected alphas:
 param_model__alpha  RMSE
               0.00 55.69
               0.01 55.66
               0.10 55.51
               1.00 55.35
              10.00 56.53
             100.00 76.72
```

Figure 17: Order in which Lasso removes features, and CV RMSE at selected α values

Figure 16 summarises this. The coefficient norm falls for all three models, but the non-zero count drops step by step only for Lasso and ElasticNet; for Ridge it stays at twelve. Figure 17 gives the order of removal: tch goes first (α ≈ 0.38), then bp, ldl_hdl_ratio, age, tc, glu, sex, ldl and hdl. The last three survivors are bmi, ltg and finally **bmi_x_bp** (α ≈ 51), matching the strongest correlations in Section 3. One subtle effect is that bmi is zero at α = 0.398 but re-enters between about 0.6 and 18 as bmi_x_bp shrinks. Lasso paths need not be monotone when features are collinear. Finally, the CV error is almost flat (55.3 to 55.7) for α between 0.001 and 1, then rises to the baseline 76.7 at α = 100 (Appendix B). Moderate regularisation therefore improves interpretability and stability more than raw accuracy.

8. Error analysis and interpretation

Figure 18 plots the single set of test predictions made in Section 6; nothing was tuned after this point. The points follow the diagonal, but the model **compresses predictions towards the mean**. Patients in the lowest quartile of actual progression are over-predicted by 44 points on average, and those in the highest quartile are under-predicted by 38.



Test error by actual-target quartile:

q	MAE	mean_residual
Q1 low	45.8	-43.9
Q2	29.3	-16.9
Q3	41.8	28.5
Q4 high	50.0	38.2

Figure 18: Predicted versus actual and residuals versus predicted on the test set, with error by target quartile

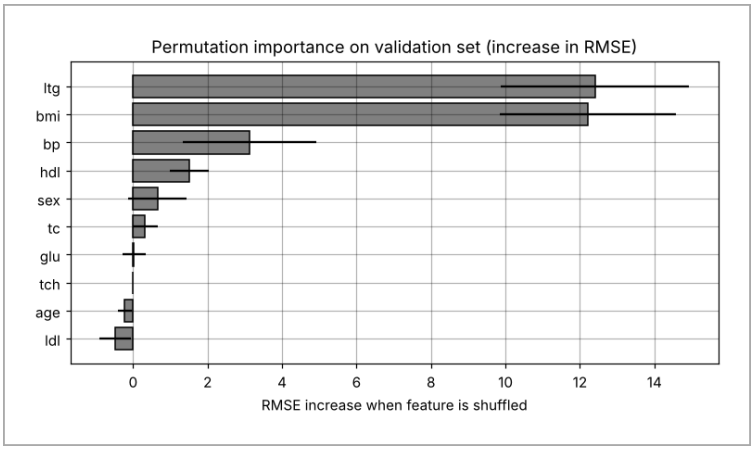


Figure 19: Permutation importance on the validation set (model fitted on training data only)

Permutation importance was computed on the **validation** set, to avoid a second look at the test set (Figure 19). Shuffling **ltg** or **bmi** raises RMSE by about 12 points each. Shuffling raw bmi also breaks bmi_x_bp, so bmi stays important even though Lasso set its own coefficient to zero. Blood pressure adds about 3 points, hdl about 1.5 and sex under 1; age, glu, tch and ldl add nothing measurable. In practice, body weight, triglycerides and blood pressure carry almost all the predictive signal.

	age	sex	bmi	bp	ltg	hdl	actual	predicted	residual
380	53.0	1.0	28.8	111.67	4.0775	46.0	52.0	175.8	-123.8
289	28.0	2.0	31.5	83.00	5.3132	38.0	68.0	186.5	-118.5
387	36.0	1.0	27.8	73.00	3.4965	42.0	201.0	93.4	107.6
190	51.0	1.0	25.2	103.00	4.8978	37.0	292.0	185.0	107.0
141	41.0	1.0	30.8	81.00	5.1358	28.0	317.0	212.4	104.6
232	52.0	2.0	26.4	91.33	4.9053	39.0	259.0	156.5	102.5
256	35.0	1.0	41.3	81.00	4.9488	37.0	346.0	244.2	101.8
112	25.0	1.0	28.3	87.00	4.3820	49.0	246.0	146.1	99.9

Median bmi / ltg in training data: 25.6 / 4.63

Figure 20: The eight test patients with the largest absolute errors

The worst cases (Figure 20) show where the model fails. Patient 380 progressed only to 52 but was predicted 176, mainly because of high blood pressure (111.7) and an above-median BMI; patient 256, with a BMI of 41.3 against a training median of 25.6, reached 346 but was predicted 244. These patients behave differently from what their baseline measurements suggest, probably because of factors the dataset does not record, such as medication, diet, adherence or HbA1c. More informative features, an interaction between ltg and bmi, or a model that predicts a range rather than one number (quantile regression) would address this better than further tuning. In short, the model is useful for ranking patients by broad risk band, but its individual predictions are too uncertain to set a precise expected score for one patient.

9. Ethics, limitations and risks

The dataset contains 442 de-identified patients from a single study, with no record of ethnicity, location, income or treatment. The model may therefore not transfer to other populations, and its errors could fall unevenly on groups that are missing from the data. The largest harm comes from under-predicting high-risk patients: the model under-estimates the top quartile by about 38 points, so a patient who most needs early care could be ranked as average. Over-prediction for low-risk patients could cause unnecessary worry or treatment. Sex is used as a feature; it adds little signal, and its inclusion should be reviewed with clinicians.

The model explains only about half of the variance, so it is suitable for research or as one input to a clinician's decision, never as an automatic triage rule. Limitations of the experiment itself include the small sample, a test set of only 89 patients, the absence of any external validation, and the use of a single random split, so the reported scores carry uncertainty of several RMSE points. Any real deployment would need privacy safeguards, prospective validation and fairness checks on a larger, more diverse cohort.

10. Conclusion and future work

Regularised linear regression worked best on this small, collinear dataset. Lasso reduced test RMSE from 77.7 for the baseline to 52.2 and explained 55% of the variance. The engineered bmi × bp feature became the strongest single coefficient. Tree ensembles did not help, and Ridge, Lasso and ElasticNet reached almost identical accuracy. Their real difference is how they shrink: Ridge spreads weight across correlated features, while Lasso selects a few. What did not work was predicting the extremes, where errors of around 100 points remain. With more time I would use nested cross-validation and bootstrap confidence intervals for the test score, try quantile regression or a log-transformed target for the tails, and test the model on an external cohort with richer clinical data.

References

[1] B. Efron, T. Hastie, I. Johnstone and R. Tibshirani, "Least angle regression," *Annals of Statistics*, vol. 32, no. 2, pp. 407–499, 2004.

[2] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[3] A. E. Hoerl and R. W. Kennard, "Ridge regression: Biased estimation for nonorthogonal problems," *Technometrics*, vol. 12, no. 1, pp. 55–67, 1970.

[4] R. Tibshirani, "Regression shrinkage and selection via the lasso," *Journal of the Royal Statistical Society, Series B*, vol. 58, no. 1, pp. 267–288, 1996.

[5] H. Zou and T. Hastie, "Regularization and variable selection via the elastic net," *Journal of the Royal Statistical Society, Series B*, vol. 67, no. 2, pp. 301–320, 2005.

[6] scikit-learn developers, "sklearn.datasets.load_diabetes," scikit-learn documentation. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_diabetes.html

[7] L. Breiman, "Random forests," *Machine Learning*, vol. 45, pp. 5–32, 2001.

[8] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.

Appendix A: Key code snippets

```
[1]  import numpy as np, pandas as pd, matplotlib.pyplot as plt, seaborn as sns, warnings
     from sklearn.datasets import load_diabetes
     from sklearn.model_selection import train_test_split, RepeatedKFold, GridSearchCV, RandomizedSearchCV, cross_
         validate
     from sklearn.pipeline import Pipeline
     from sklearn.compose import ColumnTransformer
     from sklearn.preprocessing import StandardScaler, OneHotEncoder, FunctionTransformer
     from sklearn.impute import SimpleImputer
     from sklearn.dummy import DummyRegressor
     from sklearn.linear_model import LinearRegression, Ridge, Lasso, ElasticNet
     from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
     from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
     from sklearn.inspection import permutation_importance
     import sklearn
     warnings.filterwarnings('ignore')
     SEED = 42
     pd.set_option('display.width', 110); pd.set_option('display.max_columns', 20)
     np.random.seed(SEED)
     plt.style.use('grayscale')
     plt.rcParams.update({'figure.dpi': 110, 'axes.grid': True, 'grid.alpha': 0.3,
                          'figure.facecolor': 'white', 'axes.facecolor': 'white', 'savefig.facecolor': 'white'})
     print('scikit-learn', sklearn.__version__)

     scikit-learn 1.9.1
```

Figure A1: Imports, fixed seed and greyscale plot style

```
[11] Xt = make_pipe(Ridge()).fit(X_train, y_train)[:-1].transform(X_train)
     feature_names = num_cols + ['sex_2']
     print('Transformed training matrix:', Xt.shape)
     print('Means after scaling (train):', np.round(Xt[:, :len(num_cols)].mean(0), 3))
     print('Std after scaling (train):  ', np.round(Xt[:, :len(num_cols)].std(0), 3))
     eng = add_features(X_train)
     print('Correlation of engineered features with target (train):')
     print(eng[['bmi_x_bp', 'ldl_hdl_ratio']].corrwith(y_train).round(3).to_string())

     Transformed training matrix: (264, 12)
     Means after scaling (train): [ 0. -0.  0.  0. -0.  0. -0.  0. -0. -0.  0.]
     Std after scaling (train):   [1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1.]
     Correlation of engineered features with target (train):
     bmi_x_bp         0.634
     ldl_hdl_ratio    0.257
```

Figure A2: Check that scaling is fitted on training data (mean 0, sd 1) and engineered-feature correlations

```
[18] styles = ['-', '--', '-.', ':']; greys = ['0.0', '0.35', '0.6']
     fig, axes = plt.subplots(1, 3, figsize=(15, 4.8), sharey=True)
     for ax, (k, P) in zip(axes, paths.items()):
         for j, f in enumerate(feature_names):
             ax.plot(path_alphas, P[:, j], ls=styles[j % 4], color=greys[j // 4], lw=1.4, label=f)
         ax.axvline(best_alpha[k], color='black', lw=1, ls=':'); ax.axhline(0, color='0.5', lw=0.6)
         ax.set(xscale='log', title=f'{k}\nCV-chosen alpha = {best_alpha[k]:.3g}', xlabel='alpha (log scale)')
     axes[0].set_ylabel('Standardised coefficient')
     axes[2].legend(fontsize=7, ncol=2, loc='upper right')
     plt.suptitle('Coefficient paths: shrinkage as regularisation grows')
     plt.tight_layout(); plt.savefig('fig_paths.png', dpi=150); plt.show()

     <Figure size 1650x528 with 3 Axes>
```

Figure A3: Plotting the coefficient paths with greyscale line styles

```
[20]  # alpha at which each feature first becomes exactly zero (and stays zero) on the Lasso path
      P = paths['Lasso']; drop_alpha = {}
      for j, f in enumerate(feature_names):
          nz = np.where(np.abs(P[:, j]) > 1e-6)[0]
          drop_alpha[f] = path_alphas[nz.max() + 1] if len(nz) and nz.max() + 1 < len(path_alphas) else (path_alpha
          s[0] if not len(nz) else np.inf)
      order = pd.Series(drop_alpha).sort_values()
      print('Lasso: alpha beyond which each coefficient stays exactly zero (earliest at top)')
      print('Zero at the CV-chosen alpha:', [f for f in feature_names if abs(coef_at_best.loc[f, "Lasso"]) < 1e-6])
      print(order.map(lambda v: f'{v:.3g}').to_string())
      cv_curve = pd.DataFrame(tuned['Lasso'].cv_results_)[['param_model__alpha', 'mean_test_score']]
      print('\nLasso CV RMSE at selected alphas:')
      print(cv_curve.iloc[::10].assign(RMSE=lambda d: -d.mean_test_score)[['param_model__alpha', 'RMSE']].round(2).
          to_string(index=False))
```

Figure A4: Finding the α beyond which each Lasso coefficient stays at zero

```
[22]  resid = y_test - p_test
      fig, ax = plt.subplots(1, 2, figsize=(12, 4.3))
      ax[0].scatter(y_test, p_test, s=16, c='0.3'); lim = [20, 350]; ax[0].plot(lim, lim, 'k--', lw=1)
      ax[0].set(title=f'Predicted vs actual ({best_name}, test set)', xlabel='Actual progression', ylabel='Predicte
          d', xlim=lim, ylim=lim)
      ax[1].scatter(p_test, resid, s=16, c='0.3'); ax[1].axhline(0, color='black', lw=1)
      ax[1].set(title='Residuals vs predicted (test set)', xlabel='Predicted', ylabel='Residual (actual - predicted
          )')
      plt.tight_layout(); plt.savefig('fig_resid.png', dpi=150); plt.show()
      q = pd.qcut(y_test, 4, labels=['Q1 low', 'Q2', 'Q3', 'Q4 high'])
      print('Test error by actual-target quartile:')
      print(pd.DataFrame({'q': q, 'abs_err': resid.abs(), 'bias': resid}).groupby('q', observed=True).agg(MAE=('abs
          _err', 'mean'), mean_residual=('bias', 'mean')).round(1).to_string())
```

Figure A5: Residual plots and error by target quartile

```
[23]  # importance is measured on the VALIDATION set with a copy fit on train only, so the test set is used once
      from sklearn.base import clone
      m_tr = clone(final).fit(X_train, y_train)
      pi = permutation_importance(m_tr, X_val, y_val, n_repeats=30, random_state=SEED, scoring='neg_root_mean_squar
          ed_error')
      order_i = np.argsort(pi.importances_mean)
      fig, ax = plt.subplots(figsize=(7, 4))
      ax.barh(X_val.columns[order_i], pi.importances_mean[order_i], xerr=pi.importances_std[order_i], color='0.5',
          edgecolor='black')
      ax.set(title='Permutation importance on validation set (increase in RMSE)', xlabel='RMSE increase when featur
          e is shuffled')
      plt.tight_layout(); plt.savefig('fig_importance.png', dpi=150); plt.show()
```

```
      <Figure size 770x440 with 1 Axes>
```

Figure A6: Permutation importance on the validation set with a model fitted on training data only

Appendix B: Notebook outputs

```
Diabetes Progression Regression
Applied AI/ML Mini Project | Problem ID 17 | Rachel Elizabeth Joy (2667213)

Predict a quantitative measure of diabetes disease progression one year after baseline from ten baseline measur
    ements
(sklearn.datasets.load_diabetes).

Focus requirement: compare Ridge, Lasso and ElasticNet and show how coefficients shrink as regularisation grows
    .
```

```
[25] print(f'Best model: {best_name}  |  test RMSE {test_scores["Test RMSE"]:.2f}, MAE {test_scores["Test MAE"]:.2
         f}, R2 {test_scores["Test R2"]:.3f}')
     print(f'Baseline test RMSE {mean_squared_error(y_test, base_test) ** 0.5:.2f}')
     print('Run completed without errors.')
```

```
Best model: Lasso  |  test RMSE 52.21, MAE 41.63, R2 0.548
Baseline test RMSE 77.65
Run completed without errors.
```

Figure B1: Notebook title cell and the final cell after Runtime → Run all, completed without errors

```
[3] display(data.describe().T.round(2))
    print('Missing values per column:'); print(data.isna().sum().to_string())
    print('Sex value counts:'); print(data['sex'].value_counts().to_string())
```

	count	mean	std	min	25%	50%	75%	max
age	442.0	48.52	13.11	19.00	38.25	50.00	59.00	79.00
sex	442.0	1.47	0.50	1.00	1.00	1.00	2.00	2.00
bmi	442.0	26.38	4.42	18.00	23.20	25.70	29.28	42.20
bp	442.0	94.65	13.83	62.00	84.00	93.00	105.00	133.00
tc	442.0	189.14	34.61	97.00	164.25	186.00	209.75	301.00
ldl	442.0	115.44	30.41	41.60	96.05	113.00	134.50	242.40
hdl	442.0	49.79	12.93	22.00	40.25	48.00	57.75	99.00
tch	442.0	4.07	1.29	2.00	3.00	4.00	5.00	9.09
ltg	442.0	4.64	0.52	3.26	4.28	4.62	5.00	6.11
glu	442.0	91.26	11.50	58.00	83.25	91.00	98.00	124.00
target	442.0	152.13	77.09	25.00	87.00	140.50	211.50	346.00

Figure B2: Summary statistics of every column in original units

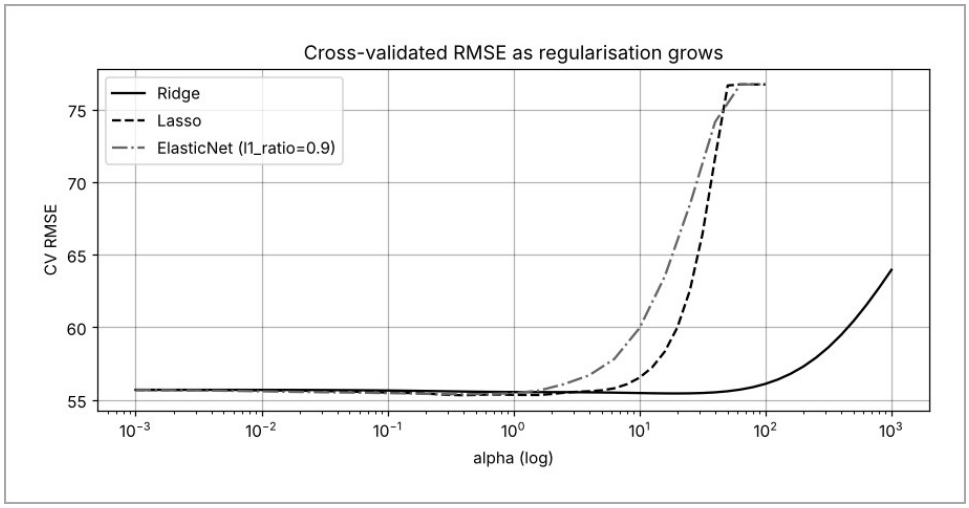


Figure B3: Cross-validated RMSE against α for Ridge, Lasso and ElasticNet

Appendix C: AI tool declaration

Table C1: AI tools used

Tool	What I used it for	What I changed or checked myself
Claude (Anthropic)	Drafting the notebook code structure, the plots and a first draft of the report text	[Describe what you checked: e.g. re-ran the notebook in Colab, verified every number in the report against the outputs, rewrote sections in my own words]